Lecture 18

March 25, 2025

Random Projections, Embeddings & Kernel Evaluation

- The need to compute kernels like $\kappa(x_i, x_j)$ plays an important role in kernel machines, like the support vector machine

- Given a large quantity of data $\{x_i\}_{i=1,N}$ the $N \times N$ *Gram Matrix* composed of all elements of $\kappa(x_i, x_j)$ expensive to compute and to store

- Prior work has considered a nonlinear and *stochastic* transformation $\phi(x)$, and

$$\kappa(x_i, x_j) = \mathbb{E}[\phi(x_i)^T \phi(x_j)]$$

- Note: The mapping $x \to \phi(x)$ is <u>stochastic</u>, and we average over random samples

- Works efficiently because often the number of samples m needed to sample over can be much smaller than the dimension of $x \in \mathbb{R}^d$

Random Features for Efficient Kernel Computation

- A. Rahimi and B. Recht, "Random Features for Large-Scale Kernel Machines," 2007

- Property of the Fourier transform of a kernel

$$\kappa(x-y) = \int_{\mathbb{R}^d} p(\omega) e^{j\omega^T(x-y)} d\omega = \mathbb{E}_\omega[\xi_\omega(x)\xi_\omega(y)^*]$$

  where $\xi_\omega(x) = \exp(j\omega^T x)$

- Therefore, $\xi_\omega(x)\xi_\omega(y)^*$ is an unbiased estimate of $k(x-y)$ when $\omega \sim p(\omega)$

- Ideally we would like to average over a small number of samples $\xi_\omega(x)\xi_\omega(y)^*$

Random Features for Efficient Kernel Computation

$$\begin{aligned}\kappa(x-y) &= \int_{\mathbb{R}^d} p(\omega) e^{j\omega^T(x-y)} d\omega = \mathbb{E}_\omega[\xi_\omega(x)\xi_\omega(y)^*] \\ \xi_\omega(x) &= cos(\omega^T x) + jsin(\omega^T x)\end{aligned}$$

- The choice of $p(\omega)$ defines the kernel
- Important example, the Gaussian kernel

$$\kappa(x-y) = exp(-\frac{\|x-y\|_2^2}{2}) \ , \qquad p(\omega) = \frac{\exp(-\frac{\|\omega\|_2^2}{2})}{\sqrt{2\pi}^D}$$

- Since $p(\omega)$ is symmetric, if $\omega \sim p(\omega)$, then $-\omega$ drawn with the same probability

Average Over Sample and Its Negative

$$\begin{aligned}\kappa(x-y) &= \int_{\mathbb{R}^d} p(\omega) e^{j\omega^T(x-y)} d\omega = \mathbb{E}_\omega[\xi_\omega(x)\xi_\omega(y)^*] \\ \xi_\omega(x) &= cos(\omega^T x) + jsin(\omega^T x)\end{aligned}$$

- We average over terms

$$\begin{aligned}\xi_\omega(x)\xi_\omega(y)^* &= [cos(\omega^T x) + jsin(\omega^T x)][cos(\omega^T y) - jsin(\omega^T y)] \\ &= cos(\omega^T x)cos(\omega^T y) + sin(\omega^T x)sin(\omega^T y) \\ &\quad + jsin(\omega^T x)cos(\omega^T y) - jsin(\omega^T y)cos(\omega^T x)\end{aligned}$$

- For the Gaussian kernel, $p(\omega)$ is symmetric, so ω and $-\omega$ equally probable, and therefore for each $\omega \sim p(\omega)$ we also consider $-\omega$ and divide sum by two

- For each $\omega \sim p(\omega)$, this yields

$$\frac{[\xi_\omega(x)\xi_\omega(y)^* + \xi_{-\omega}(x)\xi_{-\omega}(y)^*]}{2} = cos(\omega^T x)cos(\omega^T y) + sin(\omega^T x)sin(\omega^T y)$$

Setup for Kernel Computation

- Suppose we want to calculate $\exp(-\|x-y\|_2^2/2)$ with $m > 1$ random samples
- Draw $\omega_i \sim \exp(-\|\omega\|_2^2/2)/\sqrt{2\pi}^D$
- Define the feature transformations $\phi(x)$ and $\phi(y)$:

$$\begin{aligned}\phi(x) &= \frac{1}{\sqrt{m}}(cos(\omega_1^T x), \ldots, cos(\omega_m^T x), sin(\omega_1^T x), \ldots, sin(\omega_m^T x))^T \\ \phi(y) &= \frac{1}{\sqrt{m}}(cos(\omega_1^T y), \ldots, cos(\omega_m^T y), sin(\omega_1^T y), \ldots, sin(\omega_m^T y))^T\end{aligned}$$

- Then we have the approximate representation for the kernel:

$$\exp(-\frac{\|x-y\|_2^2}{2}) \approx \phi(x)^T \phi(y)$$

Leverage This Concept for Efficient Attention Computation

- We have shown that we may approximate a kernel via a stochastic transformation

$$\exp(-\frac{\|x-y\|_2^2}{2}) \approx \phi(x)^T \phi(y)$$

- If $m \ll d$, this can be more efficient that direct kernel computations

- The function $\phi(x)$ may be viewed as a fixed nonlinear transformation, but "under the hood" its components are manifested stochastically

- This idea can be leveraged and built upon for efficient computation of attention for long sequences

Random Projections for Efficient Attention for Long Sequences

- We know that context plays a vital role in the performance of Transformers
- However, large context makes the computation of all needed attention intractable if done naively
- Examine the idea of random projections for efficient kernel computation in the context of attention
- Will build naturally on the above ideas, but will need to account for special characteristics of softmax-based attention
- In softmax we must *normalize* the kernel, so direct use of above doesn't apply

Attention Setup

- Assume we have a sequence of length L and the embedding dimension is d
- We have query, key and value matrices $\mathrm{Q}, \mathrm{K}, \mathrm{V} \in \mathbb{R}^{L \times d}$
- The queries, keys and values are the *rows* of these matrices
- At this point we are looking at the attention connected to *any* attention head, so we ignore the attention-head-dependent projection matrices

Bidirection Attention

- Bidirectional attention is specified as

$$
\begin{aligned}
\mathrm{Att}_{\leftrightarrow}(\mathrm{Q}, \mathrm{K}, \mathrm{V}) &= \mathrm{D}^{-1}\mathrm{AV} \\
\mathrm{A} &= \exp(\mathrm{QK}^T/\sqrt{d}) \\
\mathrm{D} &= \mathrm{diag}(\mathrm{A}1_L)
\end{aligned}
$$

  where 1_L is an L-dimensional vector of all-ones

- Important:
  - Direct storage of attention matrix A requires $O(L^2)$ storage
  - Direct computation has $O(L^2)$ complexity
  - Significantly limits context length, but context key to Transformer performance

Unidirection (Auto-Regressive) Attention

- In models like GPT we only attend to tokens that come prior to the current token of interest (also called masked attention)

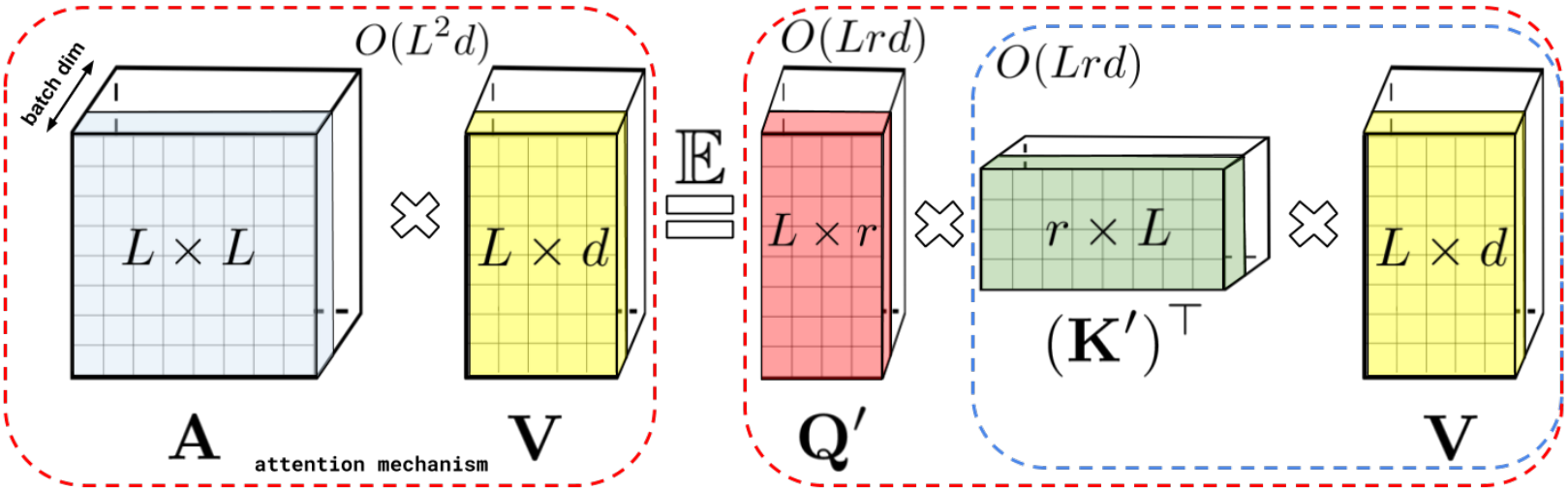
$$
\begin{aligned}
\mathrm{Att}_{\rightarrow}(Q, K, V) &= \tilde{D}^{-1}\tilde{A}V \\
\tilde{A} = \mathrm{tril}(A)\ , \qquad A &= \exp(QK^T/\sqrt{d}) \\
\tilde{D} &= \mathrm{diag}(\tilde{A}1_L)
\end{aligned}
$$

where $\mathrm{tril}(\cdot)$ represents the lower-triangle part of the input matrix (imposes autoregressive or causal attention)

$$
\mathbf{M} = \begin{bmatrix} a_{11} & 0 & 0 & 0 & 0 \\ a_{21} & a_{22} & 0 & 0 & 0 \\ a_{31} & a_{32} & a_{33} & 0 & 0 \\ a_{41} & a_{42} & a_{43} & a_{44} & 0 \\ a_{51} & a_{52} & a_{53} & a_{54} & a_{55} \end{bmatrix}_{5 \times 5}
$$

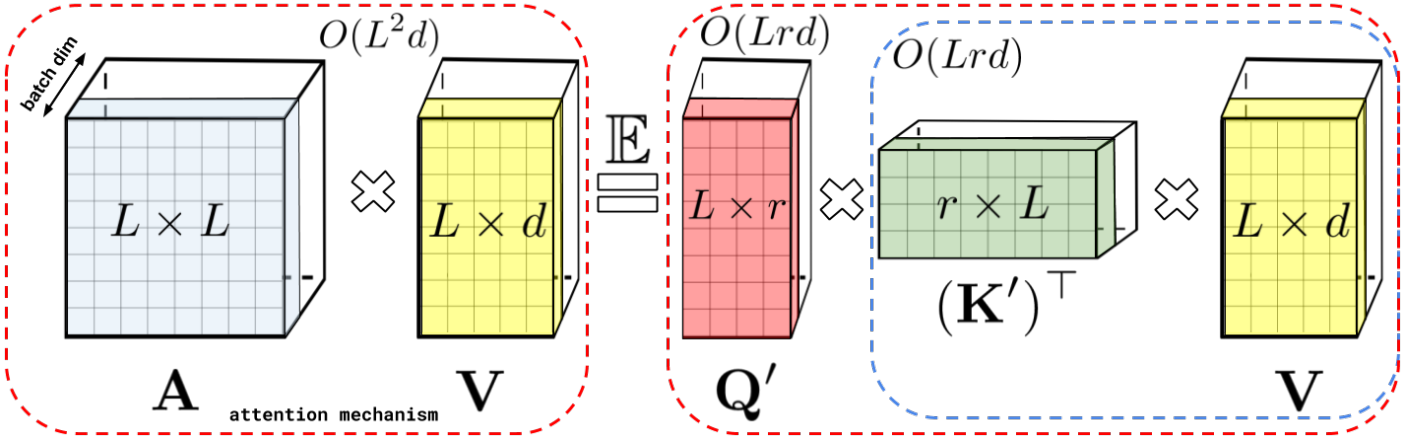
Efficient Computations

- Assume that we can *map* the d-dimensional keys and queries into a $r < d$ dimensional space: $q \to \phi(q)$, $k \to \phi(k)$
- Refer to the matrices of $\phi(q)$ and $\phi(k)$ as $\mathrm{Q}', \mathrm{K}' \in \mathbb{R}^{L \times r}$
- Break the needed computations into blocks:



- Note that $r < d$

Important Observation



- The attention matrix A must be exponentiated everywhere, and then normalized (left hand side of figure)
- We never fully form the $L \times L$ matrix A, and $\exp(\cdot)$ not evident, but $\exp(\cdot)$ will be accounted for in design of random features
- Is it possible to design Q' and K' such that their product is equivalent to $L \times L$ inner products, and also exponentiation?
- This is the key idea: Design Q', K' such that

$$[\mathrm{Q}'(\mathrm{K}')^T](i,j) \approx \exp\left\{[\mathrm{Q}(\mathrm{K})^T](i,j)\right\}$$

Key Idea

$$[Q'(K')^T](i,j) \approx \exp\left\{[Q(K)^T](i,j)\right\}$$

- Will design $Q \to Q'$ and $K \to K'$ such that inner products of latter are equivalent to exponentiated inner products of former
- Take a moment to think about normalization term, which we also need

$$D = \mathrm{diag}(A1_L) \approx \mathrm{diag}(Q' \underbrace{((K')^T 1_L)}_{r\text{-dim vector output}})$$

Key Idea

$$[Q'(K')^T](i,j) \approx \exp\left\{[Q(K)^T](i,j)\right\}$$

- Will design $Q \to Q'$ and $K \to K'$ such that inner products of latter are equivalent to exponentiated inner products of former
- Take a moment to think about normalization term, which we also need

$$D = \mathrm{diag}(A1_L) \approx \mathrm{diag}(Q' \underbrace{((K')^T 1_L)}_{O(Lr) \text{ comp, once}})$$

Key Idea

$$[\mathrm{Q}'(\mathrm{K}')^T](i,j) \approx \exp\left\{[\mathrm{Q}(\mathrm{K})^T](i,j)\right\}$$

- Will design $\mathrm{Q} \to \mathrm{Q}'$ and $\mathrm{K} \to \mathrm{K}'$ such that inner products of latter are equivalent to exponentiated inner products of former
- Take a moment to think about normalization term, which we also need

$$\mathrm{D} = \mathrm{diag}(\mathrm{A}1_L) \approx \mathrm{diag}(\ \underbrace{\mathrm{Q}'((\mathrm{K}')^T 1_L)}_{\mathrm{O}(Lr)\text{ comp, once}}\ )$$

- Total cost of diagonal term $O(Lr)$ instead of $O(L^2)$
- Will use similar sequential computations to compute other aspects of attention

Our Objective

$$[\mathrm{Q}'(\mathrm{K}')^T](i,j) \approx \exp\left\{[\mathrm{Q}(\mathrm{K})^T](i,j)\right\}$$

- Design the mapping $\mathrm{Q} \to \mathrm{Q}'$ and $\mathrm{K} \to \mathrm{K}'$ such that inner products of latter are equivalent to exponentiated inner products of former
- The exponentiated inner product is one type of kernel
- The basic construction could possibly be generalized such that this may be applied to other kernels, beyond exponentiated inner product
- Let $\kappa(x, y)$ be a kernel for two vectors x and y
- Examples:

$$\kappa(x, y) = \exp[x^T y] \ , \qquad \kappa(x, y) = \exp[-\|x - y\|_2^2/\sigma^2]$$

Basic Idea

- Like at the start, consider a <u>stochastic</u> mapping $\phi(x) \in \mathbb{R}^r$
- Specify $\phi(x)$ such that

$$\kappa(x, y) = \mathbb{E}[\phi(x)^T \phi(y)]$$

  where expectation is over aforementioned stochasticity
- The rows of $\mathsf{Q}', \mathsf{K}' \in \mathbb{R}^{L \times r}$ defined by $\phi(x)$
- Different choices of $\phi(x)$ yield different kernels $\kappa(x, y)$
- In practice we do not explicitly perform expectation, the multiple random samples built into $\phi(x)$, in the sense discussed above

Design Idea Number 1

- Consider functions $f_1, \ldots, f_l : \mathbb{R} \to \mathbb{R}$
- Function $h : \mathbb{R}^r \to \mathbb{R}$
- Deterministic vectors ω_i or $\omega_i \overset{\text{iid}}{\sim} \mathcal{D}$ for some distribution on $\mathbb{R}^d$; constitute $\{\omega_i\}_{i=1,m}$
- Define the mapping

$$\phi(x) = \frac{h(x)}{\sqrt{m}}(f_1(\omega_1^T x), \ldots, f_1(\omega_m^T x), \ldots, f_l(\omega_1^T x), \ldots, f_l(\omega_m^T x))$$

- Different settings may model most kernels used in practice

Reminiscent of Setup for Gaussian Kernel

- For Gaussian kernel $f_1(\cdot) = cos(\cdot)$, $f_2(\cdot) = sin(\cdot)$ and $h(x) = 1$
- Draw $\omega_i \sim \exp(-\|\omega\|_2^2)/\sqrt{2\pi}^D$
- Define the feature transformations $\phi(x)$ and $\phi(y)$:

$$\begin{aligned}\phi(x) &= \frac{1}{\sqrt{m}}(cos(\omega_1^T x), \ldots, cos(\omega_m^T x), sin(\omega_1^T x), \ldots, sin(\omega_m^T x))^T \\ \phi(y) &= \frac{1}{\sqrt{m}}(cos(\omega_1^T y), \ldots, cos(\omega_m^T y), sin(\omega_1^T y), \ldots, sin(\omega_m^T y))^T\end{aligned}$$

- Then we have the approximate representation for the kernel:

$$\exp(-\frac{\|x-y\|_2^2}{2}) \approx \phi(x)^T \phi(y)$$

Closer Look at this Generalization

$$\phi(x) = \frac{h(x)}{\sqrt{m}}(f_1(\omega_1^T x), \ldots, f_1(\omega_m^T x), \ldots, f_l(\omega_1^T x), \ldots, f_l(\omega_m^T x))$$

- The kernel is

$$\begin{aligned}
\kappa(x, y) &= \phi(x)^T \phi(y) \\
&= h(x)h(y) \sum_{j=1}^{l} \frac{1}{m} \sum_{i=1}^{m} f_j(\omega_i^T x) f_j(\omega_i^T y) \\
&\approx h(x)h(y) \sum_{j=1}^{l} \mathbb{E}_\omega \left[f_j(\omega^T x) f_j(\omega^T y) \right]
\end{aligned}$$

Gaussian and Softmax Kernels

- Consider

$$h(x) = 1, \quad l = 2, \quad f_1(u) = \sin(u), \quad f_2(u) = \cos(u), \quad \mathcal{D} = \mathcal{N}(0, \mathrm{I}_d)$$

- This design yields an approximation to

$$\kappa_{\text{gauss}}(x, y) = \exp(-\frac{1}{2}\|x-y\|_2^2) = \exp[-\frac{1}{2}\|x\|_2^2 - \frac{1}{2}\|y\|_2^2 + x^T y]$$

- From $\kappa_{\text{gauss}}(x, y)$, define the kernel needed for softmax as

$$\begin{aligned} \kappa_{\text{SM}}(x, y) &= \exp(x^T y) \\ &= \exp(\frac{\|x\|_2^2}{2})\kappa_{\text{gauss}}(x, y)\exp(\frac{\|y\|_2^2}{2}) \end{aligned}$$

- Note that $\kappa_{\text{SM}}(x, y)$ is the *unnormalized* softmax attention

Gaussian and Softmax Kernels

- The explicit form of the functional mapping is

$$\phi(x) = \frac{exp(\frac{\|x\|_2^2}{2})}{\sqrt{m}}(\sin(\omega_1^T x), \ldots, \sin(\omega_m^T x), \cos(\omega_1^T x), \ldots, \cos(\omega_m^T x))$$

- With this definition of $\phi(x)$,

$$\kappa_{\mathrm{SM}}(x, y) = \exp(x^T y) = \phi(x)^T \phi(y)$$

- Problem: $\phi(x)$ has components with positive and negative values

- Leads to unstable performance of this framework (high variance), particularly for small values of $\kappa_{\mathrm{SM}}(x, y)$

Design Idea Number 2

- Motivates a new framework in which all components of $\phi(x)$ are positive

$$\begin{aligned}\kappa_{\mathrm{SM}}(x,y) &= \mathbb{E}_{\omega\sim\mathcal{N}(0,\mathrm{I}_d)}\Big[\exp(\omega^T x - \frac{\|x\|_2^2}{2})\exp(\omega^T y - \frac{\|y\|_2^2}{2})\Big] \\ &= \exp\big(-\frac{\|x\|_2^2+\|y\|_2^2}{2}\big)\mathbb{E}_{\omega\sim\mathcal{N}(0,\mathrm{I}_d)}\cosh[\omega^T(x+y)]\end{aligned}$$

- This approach builds on the original idea from the Gaussian kernel, but it has the advantage that every added random variable is positive

- This has the advantage of reducing the variance of the estimator

Design Idea Number 2

- Motivates a new framework in which all components of $\phi(x)$ are positive

$$\begin{aligned}\kappa_{\mathrm{SM}}(x,y) &= \mathbb{E}_{\omega\sim\mathcal{N}(0,\mathrm{I}_d)}\Big[\exp(\omega^T x - \frac{\|x\|_2^2}{2})\exp(\omega^T y - \frac{\|y\|_2^2}{2})\Big] \\ &= \exp\big(-\frac{\|x\|_2^2+\|y\|_2^2}{2}\big)\mathbb{E}_{\omega\sim\mathcal{N}(0,\mathrm{I}_d)}\cosh[\omega^T(x+y)]\end{aligned}$$

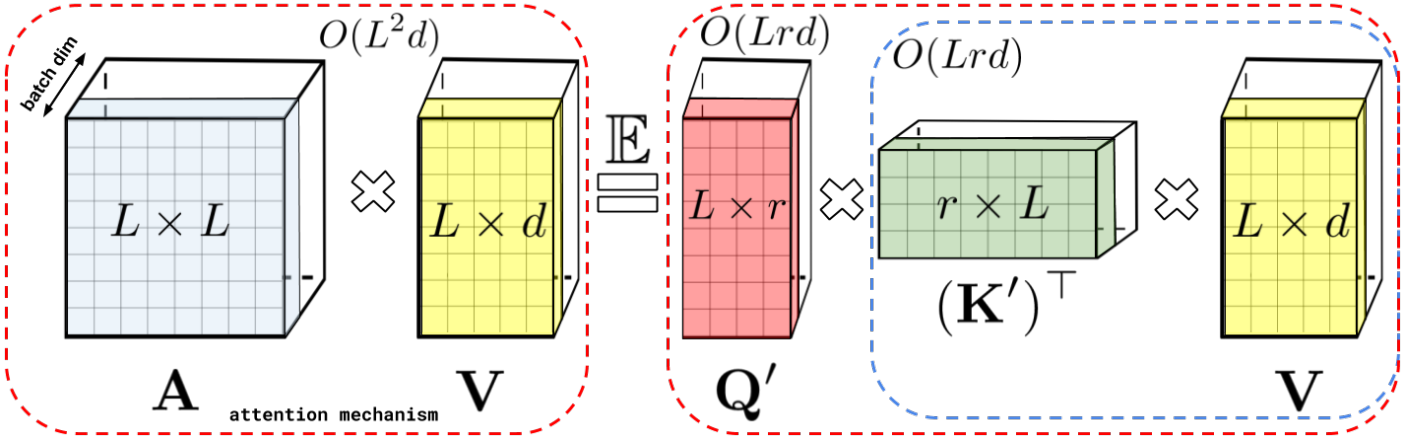
- Yields following designs for $\kappa_{\mathrm{SM}}(x,y)$: $\mathcal{D}=\mathcal{N}(0,\mathrm{I}_d)$ and $l=2$, with

$$h(x)=\frac{1}{\sqrt{2}}\exp\big(\frac{-\|x\|_2^2}{2}\big)\,,\quad f_1(u)=\exp(u)\,,\quad f_2(u)=\exp(-u)$$

Orthogonal Random Features

- To further reduce variance of the estimator, the RVs $\omega_1, \ldots, \omega_m$ are "disentangled" to be **exactly** orthogonal
- Standard Gram-Schmidt orthogonalization on m RVs
- Reduction in variance reduces the needed number of RVs, and hence the dimension r
- Yields further acceleration
- Method: Fast Attention Via positive Orthogonal Random features (FAVOR+)

Summary



- For long sequence lengths L, we do not directly compute the attention matrix A
- Via a novel random projection $x \to \phi(x)$ the exponentiated matrix is computed as matrix product of random variables
- Computational cost goes from $O(L^2)$ to $O(Lr)$
- Design of random projections makes $r \ll L$
- Same idea used for normalization $\mathrm{diag}(\mathrm{A}1_L)$

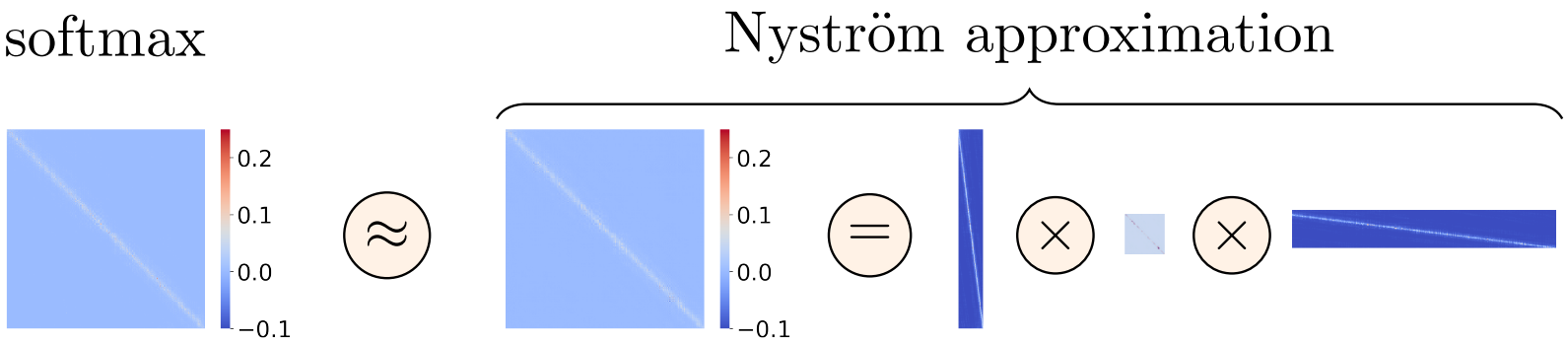
Other Approaches for Attention with Long Sequences

- The approach developed above builds on extensive prior work in the kernel literature based on random feature transformation
- The idea of random projections has a long history in machine learning, connected to low-dimensional embeddings with random projections, specifically Johnson-Lindenstrauss transformations (very famous)
- Random projections have also played an important role in fields like compressive sensing
- However, efficient kernel computations has been an area of extensive research, and other approaches have been considered

Nystrom's Method

- If the sequence length we wish to process with the Transformer is of length L, we must compute an $L \times L$ attention matrix, as discussed above
- This is caused by the need to compute attention for each of the L queries and keys
- The key idea with the Nystrom method is that maybe we do not need to compute all of these L^2 attention calculations directly
- Select a (small) set of "landmark" queries and keys, where the number of landmarks is much less than L
- Compute all landmark-landmark attentions rigorously, and develop an efficient method to extrapolate to compute all other attention measures

Main Idea



- The small square matrix computes all attention interactions between the small number of landmarks
- We then consider the landmarks as queries as applied to all of the original keys (rectangular matrix)
- Consider all of the original elements as queries, where only the landmarks are keys (the other rectangular matrix)
- Use these three matrices, which can be computed efficiently, to approximate all other elements of attention matrix

Summary of Algorithm

Algorithm 1: Pipeline for Nyström approximation of softmax matrix in self-attention

Input: Query Q and Key K.
Output: Nyström approximation of softmax matrix.
Compute landmarks from input Q and landmarks from input K, $\tilde{Q}$ and $\tilde{K}$ as the matrix form ;
Compute $\tilde{F} = \text{softmax}(\frac{Q\tilde{K}^T}{\sqrt{d_q}})$, $\tilde{B} = \text{softmax}(\frac{\tilde{Q}K^T}{\sqrt{d_q}})$;
Compute $\tilde{A} = \text{softmax}(\frac{\tilde{Q}\tilde{K}^T}{\sqrt{d_q}})^+$;
return $\tilde{F} \times \tilde{A} \times \tilde{B}$;

Summary on Efficient Attention Computation

- If done exactly, the memory and computational cost for exact attention calculation is $O(L^2)$ for sequences of length L
- This limits the amount of context that may be considered by a Transformer
- Things like chain-of-thought prompting and RAG tell us that appropriate use of extensive context can markedly impact Transformer performance
- Therefore leverage approximate methods that can yield $O(L)$ computational and memory cost, at minimal loss of accuracy
- Leverage extensive prior work in the ML literature, from the days of kernel machines, e.g., random projections and Nystrom methods of extrapolation

Remainder of the Semester

- Lecture 19: Mixtures of Experts and The Transformer
- Lecture 20: The Vision Transformer
- Lecture 21: Cross-Modal Learning with Transformers
- Lecture 22: Diffusion and The Transformer
- lecture 23: The Graph Transformer
- Lecture 24: Recap